تطبيقات الإنترنت والمنصات الإلكترونية

JavaScript

كيف تتفاعل المواقع مع ضغطات المستخدم

الاسم: شروق مصطفى محمد

الرقم الجامعي: 2500421

السكشن: 81

عنوان البحث: JavaScript وتفاعل المواقع مع المستخدم

المادة: تطبيقات الإنترنت والمنصات الإلكترونية

1. مقدمة

JavaScript هي لغة البرمجة الأساسية للويب. ظهرت عام 1995 على يد Brendan Eich في شركة Netscape لتجعل الصفحات تتفاعل مع المستخدم، وكانت تُسمى في البداية Mocha ثم LiveScript قبل أن تستقر على اسمها الحالي. وتم توحيد قواعدها لاحقاً في معيار ECMAScript الذي تصدره منظمة Ecma International، وأهم إصدار حديث فيه هو ES6 (2015) الذي أضاف `let` و `const` والدوال السهمية والـ Promises والـ Classes.

عند تصفح أي موقع نجد ثلاث تقنيات تعمل معاً: HTML لبناء هيكل الصفحة ومحتواها، وCSS لتنسيق الشكل والألوان والخطوط، وJavaScript لإضافة السلوك والتفاعل. وتعمل JavaScript داخل المتصفح (Browser) على جهاز المستخدم، ويدعمها كل متصفح حديث دون تثبيت أي برنامج. وأصبحت تعمل أيضاً خارج المتصفح على الخوادم عبر Node.js (ظهر عام 2009)، لذلك تُستخدم في المواقع والتطبيقات والألعاب وتطبيقات الموبايل.

تتناول هذه الورقة أساسيات اللغة، ثم كيف تتعامل مع الصفحة (DOM) وتستجيب لضغطات المستخدم (Events)، مع أمثلة عملية.

2. أساسيات لغة JavaScript

2.1 كيف نضيف JavaScript إلى الصفحة؟

بثلاث طرق: داخل وسم `<script>` في نفس الملف، أو في ملف خارجي بامتداد `.js` يُربط بالصفحة (وهي الطريقة الأفضل لتنظيم الكود)، أو داخل خاصية الحدث مباشرة وهي طريقة غير مفضلة.

```
<script src="script.js"></script>   <!-- external file, placed before </body> -->
```

2.2 المتغيرات وأنواع البيانات والشروط والحلقات

المتغير (Variable) مكان لتخزين قيمة. نستخدم `let` للقيمة التي تتغير و `const` للقيمة الثابتة، ونتجنب `var` القديمة. وأهم أنواع البيانات: النص String والرقم Number والقيمة المنطقية Boolean (`true` أو `false`) والمصفوفة Array والكائن Object. وللمقارنة نستخدم `===` لأنها تقارن القيمة والنوع معاً.

```
let name = "Sara";          // String
const PI = 3.14;            // constant
let age = 20;               // Number
let isStudent = true;       // Boolean

if (age >= 18) { console.log("Adult"); } else { console.log("Minor"); }

for (let i = 1; i <= 3; i++) { console.log(i); }   // 1 2 3
```

2.3 الدوال (Functions)

الدالة مجموعة أوامر لها اسم نكتبها مرة ونستدعيها كلما احتجنا إليها، وقد تستقبل مدخلات (Parameters) وترجع نتيجة (return). وتوجد صيغة مختصرة تسمى الدالة السهمية (Arrow Function) وهي شائعة جداً في الأحداث.

```
function add(a, b) { return a + b; }
const mul = (a, b) => a * b;          // arrow function
console.log(add(2, 3), mul(2, 3));    // 5 6
```

2.4 المصفوفات والكائنات (Arrays & Objects)

المصفوفة تخزن قائمة قيم مرتبة، والكائن يخزن بيانات على شكل أزواج (اسم: قيمة). وهما أساس التعامل مع البيانات في أي موقع، مثل قائمة المنتجات في المتجر.

```
const fruits = ["apple", "banana"];
fruits.push("orange");                    // add item
fruits.forEach(f => console.log(f));      // loop over items

const student = { name: "Sara", grade: 90 };
console.log(student.name);                // Sara
```

2.5 ميزات حديثة من ES6

أضاف إصدار ES6 أدوات تجعل الكود أقصر وأوضح: النص المركب (Template Literals) بعلامة الاقتباس المائلة backtick لدمج المتغيرات داخل النص، والدالتان `map()` و `filter()` لتحويل المصفوفات وتصفيتها، والتفكيك (Destructuring) لاستخراج قيم من كائن، ومعامل النشر (Spread) `...` لنسخ المصفوفات ودمجها. ومن المهم معرفة أن `let` و `const` لهما نطاق (Scope) محدود بالقوسين `{ }` الموجودين فيهما.

```
const user = "Sara";
console.log(`Hello, ${user}!`);                    // Hello, Sara!

const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);               // [2, 4, 6, 8]
const even = nums.filter(n => n % 2 === 0);         // [2, 4]
const more = [...nums, 5];                          // [1, 2, 3, 4, 5]
const { name, grade } = { name: "Sara", grade: 90 };   // name = "Sara"
```

3. التفاعل مع الصفحة

3.1 نموذج الصفحة (DOM)

يحوّل المتصفح كود HTML إلى شجرة من العناصر تسمى DOM (اختصار Document Object Model)، ومن خلالها تصل JavaScript إلى أي عنصر لتقرأه أو تعدله. أهم الدوال: `getElementById()` و `querySelector()` للوصول للعنصر، و `textContent` لتغيير النص، و `style` لتغيير التنسيق، و `classList` لإضافة وإزالة الأصناف، و `createElement()` و `appendChild()` لإنشاء عناصر جديدة وإضافتها.

3.2 الأحداث (Events) والمستمعات (Event Listeners)

الحدث (Event) هو أي شيء يحدث في الصفحة بسبب المستخدم أو المتصفح. ويحدث التفاعل في ثلاث خطوات: (1) اختيار العنصر من (2) DOM. (3) ربطه بمستمع عبر `addEventListener()`. تنفيذ دالة عند وقوع الحدث. ومن أشهر الأحداث:

- `click`: الضغط على عنصر. `dblclick`: ضغطتان متتاليتان.
- `input` و `change`: الكتابة في حقل أو تغيير اختيار.
- `submit`: إرسال نموذج (Form). `keydown`: الضغط على مفتاح.
- `mouseover`: مرور الفأرة فوق عنصر. `load`: انتهاء تحميل الصفحة.

وتستقبل الدالة كائن الحدث (غالباً باسم e) الذي يحمل معلومات مثل العنصر الذي ضغط عليه المستخدم (`e.target`)، ويمكن منع السلوك الافتراضي مثل إرسال النموذج باستخدام `e.preventDefault()`.

3.3 انتشار الحدث (Bubbling) وتفويض الأحداث (Delegation)

عند الضغط على عنصر داخلي يصعد الحدث تلقائياً إلى العناصر الأب، وهذا يسمى Event Bubbling. ونستفيد منه في Event Delegation: نضع مستمعاً واحداً على العنصر الأب بدل مستمع لكل ابن، فيعمل أيضاً على العناصر التي تُضاف لاحقاً، وهذا أفضل أداءً.

```
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    e.target.classList.toggle("done");   // works for any <li>, even new ones
  }
});
```

4. أمثلة تطبيقية مهمة

مثال 1: عداد يزيد مع كل ضغطة

```
<p>Count: <span id="n">0</span></p>
<button id="add">+1</button>
<script>
  let count = 0;
  document.getElementById("add").addEventListener("click", () => {
    count++;
    document.getElementById("n").textContent = count;
  });
```

```
</script>
```

النتيجة: كل ضغطة تزيد الرقم واحداً (1 ثم 2 ثم 3...) لأن المتغير count يحتفظ بقيمته بين الضغطات.

مثال 2: الوضع الليلي (Dark Mode)

```
/* CSS */  body.dark { background: #111; color: #fff; }

<button id="mode">Change Theme</button>
<script>
  document.getElementById("mode").addEventListener("click", () => {
    document.body.classList.toggle("dark");
  });
</script>
```

النتيجة: الضغطة الأولى تحول الصفحة للوضع الداكن والثانية تعيدها كما كانت، لأن `toggle` تضيف الصنف أو تزيله بالتبادل.

مثال 3: قائمة مهام (To-Do List)

```
<input id="task" placeholder="New task">
<button id="addTask">Add</button>
<ul id="list"></ul>
<script>
  document.getElementById("addTask").addEventListener("click", () => {
    const input = document.getElementById("task");
    if (input.value.trim() === "") return;      // ignore empty input
    const li = document.createElement("li");    // create new element
    li.textContent = input.value;
    document.getElementById("list").appendChild(li);
    input.value = "";                           // clear the field
  });
</script>
```

النتيجة: كل ما يكتب المستخدم مهمة ويضغط Add تظهر في القائمة فوراً، وهذه فكرة إضافة المنتجات للسلة في المتاجر.

مثال 4: التحقق من نموذج (Form Validation)

```
<form id="form"><input id="email"><button>Send</button></form>
<script>
  document.getElementById("form").addEventListener("submit", (e) => {
    const email = document.getElementById("email").value;
    if (!email.includes("@")) {
      e.preventDefault();                       // stop sending
      alert("Please enter a valid email");
    }
  });
</script>
```

النتيجة: إذا كتب المستخدم بريداً بدون @ يتوقف الإرسال وتظهر رسالة تنبيه، فنوفر على الخادم استقبال بيانات خاطئة.

مثال 5: حفظ اختيار المستخدم (localStorage)

```
if (localStorage.getItem("theme") === "dark") {
  document.body.classList.add("dark");              // restore saved choice
}
document.getElementById("mode").addEventListener("click", () => {
  document.body.classList.toggle("dark");
  const isDark = document.body.classList.contains("dark");
  localStorage.setItem("theme", isDark ? "dark" : "light");   // save choice
});
```

النتيجة: يبقى الوضع الداكن مفعلاً حتى بعد تحديث الصفحة أو إغلاق المتصفح، لأن localStorage يخزن بيانات نصية (مفتاح وقيمة) داخل المتصفح.

5. مفاهيم متقدمة ومهمة

- البرمجة غير المتزامنة (Asynchronous): بعض المهام تستغرق وقتاً مثل جلب بيانات من الإنترنت، فلا تنتظرها JavaScript بل تكمل العمل وترجع لها عند الانتهاء، وبذلك لا تتجمد الصفحة. وأدواتها: `setTimeout` و `fetch` و `Promise` و `async/await`.

```
setTimeout(() => console.log("after 2 seconds"), 2000);

async function getPosts() {
  const res = await fetch("https://api.example.com/posts");   // request data
  const data = await res.json();                              // convert JSON
  console.log(data);
}
```

- JSON: صيغة نصية خفيفة لتبادل البيانات بين المتصفح والخادم، وتشبه الكائنات في JavaScript.
- أدوات المطور (Developer Tools): تُفتح بالضغط على F12، وتعرض رسائل `console.log()` والأخطاء وتتيح فحص DOM وتجربة الكود.
- معالجة الأخطاء (Error Handling): نستخدم `try / catch` حتى لا يتوقف البرنامج كله عند حدوث خطأ.

```
try {
  const data = JSON.parse(text);         // may throw an error if text is invalid
} catch (err) {
  console.log("Error:", err.message);    // program continues running
}
```

- الفرق بين JavaScript وJava: اسمهما متشابه فقط، فهما لغتان مختلفتان تماماً في الاستخدام والقواعد.
- منظومة اللغة (Ecosystem): Node.js لتشغيلها على الخادم، وnpm لإدارة المكتبات، وأطر مثل React وVue وAngular لبناء واجهات كبيرة.
- الأمان (Security): لا تُدخل نص المستخدم في الصفحة عبر `innerHTML` لأنه قد يسمح بهجمات XSS، والأفضل استخدام `textContent`. والتحقق من المدخلات في المتصفح لا يكفي، فيجب التحقق منها على الخادم أيضاً.
- أفضل الممارسات: استخدام `const` و `let`، وتسمية المتغيرات بأسماء واضحة، وكتابة تعليقات `//`، وفصل الكود في ملف `js.` مستقل، والتحقق من مدخلات المستخدم.

6. الخاتمة

JavaScript هي العنصر الذي يحوّل صفحات الويب من مستندات ثابتة إلى تطبيقات تفاعلية. فهي لغة تعتمد على المتغيرات والدوال والشروط لبناء المنطق، وعلى DOM للوصول إلى عناصر الصفحة، وعلى Events لتستجيب لما يفعله المستخدم، وعلى async/await للتعامل مع البيانات القادمة من الإنترنت. وفهم هذه الأفكار هو الأساس لأي مطور مبتدئ، وهو الطريق للانتقال لاحقاً إلى أطر مثل React وVue.

المراجع

- MDN Web Docs, "JavaScript". Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDN Web Docs, "Introduction to events". Mozilla. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/Events
- The Modern JavaScript Tutorial. javascript.info. https://javascript.info
- W3Schools, "JavaScript Tutorial". https://www.w3schools.com/js/
- Ecma International, "ECMA-262: ECMAScript Language Specification". https://ecma-international.org/publications-and-standards/standards/ecma-262/
- Haverbeke, M., "Eloquent JavaScript" (4th ed.), No Starch Press, 2024. https://eloquentjavascript.net
- Flanagan, D., "JavaScript: The Definitive Guide" (7th ed.), O'Reilly Media, 2020.